Design note

Retrieval

Policies are a small, closed set (one per retailer and category), so retrieval is a deterministic SQL lookup: `WHERE retailer = ? AND category = ?`. The retailer comes from the API key, so isolation is part of the query. I did not use a vector database or agent framework; they add nondeterminism and cannot beat an exact key lookup here. Citations are whole sentences sliced from the stored policy text, so they are verbatim by construction, and each one names the sentence that supports the decision.

Policy enforcement

Decisions are made in code (`app/engine.py`), not by the model. The thresholds are constants: returns 30 days (Northstar) and 14 (Cedar), a photo within 7 days for Northstar damage, and a carrier investigation at 6 days overdue (Northstar) or 4 (Cedar). Cedar damage is always human review. The model only rewords a reply the engine already chose. It never sees the customer's message, only the structured decision, so message text cannot inject instructions. Its output is validated for length (400 characters), sentence count (3), promises such as "we will refund", and mentions of another retailer.

ERP reconciliation

The ERP is the system of record. For each fact relevant to the ticket's category: the ERP value wins; a client value that differs is recorded as a discrepancy and the ticket goes to human review; the client can only fill a fact the ERP leaves blank, and that fact is labelled "client" in `fact_sources`. Blank or unparsable ERP fields are treated as unknown, never invented. If the ERP says an order is undelivered but the client claims a delivery age, that is a discrepancy. A delivered order always has `days_overdue = 0`, so a late-but-delivered order never triggers the shipping policy. An unknown order, or another retailer's order, returns the same `needs_information` response, so nothing is disclosed.

Authentication

`X-API-Key` maps to a retailer on the server (constant-time comparison). Any `retailer` field in the body is ignored. A missing or invalid key returns 401, before body validation (so 401 wins over 422).

Fallback

The ERP call (2 s) and the model call (8 s) are time-bounded. ERP failure, model timeout, a malformed or unsafe draft, or a missing policy produces `human_review` with no citations and no invented facts, and `fallback = true`. The audit row is still written. The audit log stores metadata only, never the customer message.

Assumptions

- "Today" is fixed at 2025-09-01. Delivery day is day zero.
- A denial that follows policy (window passed, final sale) is `policy_supported`; nothing is executed.
- Ambiguous requests and mixed issues are classified `other` and go to human review.
- An unknown order gives `needs_information`; I do not decide on client facts alone.
- `fallback` is true whenever the fallback path fired, including ERP failure.
- The model is called only for `policy_supported` and `needs_information` outcomes.

Limitations

- Rule-based classifier: accuracy 0.78 on 32 examples, all 7 misses are paraphrases.
- Day-count mismatches are compared exactly, with no tolerance.
- Injection, other-retailer and promise checks are coarse regexes.
- No retries or circuit breaker; a timed-out worker thread cannot be killed.
- The real-model provider and the Postgres/Azure SQL move are untested.

Next improvement

Add a TF-IDF plus logistic regression classifier, or an LLM for ambiguous tickets with schema-validated output, and compare both against the rule baseline on a larger labelled set. Then add async I/O, a `fallback_reason` audit column, and retries with a breaker.